

Experiment-Overview 2026-05-23

Experiment-Overview: TDD-Workflows × Modelle × Prompt-Stile

Stand: 2026-05-23. Datenbasis: `experiments/runs/` (659 Runs gesamt).

Autor: Marco Emrich (codecentric AG) — Mit-Initiator von [EXACT Coding](#) gemeinsam mit Ferdinand Ade.

Repository: github.com/marcoemrich/agentic_coding_lab — alle Skripte, Workflow-Definitionen, Run-Artefakte und das Stylesheet sind dort öffentlich versioniert.

Über die Studie

Das Agentic Coding Lab ist die empirische Validierungs-Plattform für **EXACT Coding** (**EX**ample-guided **AI**-Collaborative **T**est-driven Coding) — einen Software-Craft-Workflow für Agentic Development, der bewährte Praktiken (TDD, Example-Driven Development, Refactoring, Human-in-the-Loop) gezielt zusammenstellt, um den von Vibe Coding bekannten Qualitätsverlust zu vermeiden (siehe Manuskript `exact-coding-book/manuscript/exact-coding.md`). Die hier untersuchten Workflows decken den Bereich von Vibe-Coding-Baselines (v1/v2) bis zu vollständig EXACT-konformen TDD-Aufbauten mit isolierten Phasen-Subagents (v4) und Hybriden mit isoliertem Refactor-Subagent (v6/v7) ab; die v8-Varianten testen ein „End-Refactor nach Vibe-Coding“-Gegenmodell.

Dieser Snapshot fasst den Stand zum 2026-05-23 über 9 aktive generische Forschungsfragen in `research/questions/` zusammen — insgesamt 659 Runs in `experiments/runs/`. Die zentrale Front liegt bei der Validierung eines Hybrid-Workflows (Skill-basiertes Red/Green im geteilten Kontext + isolierter Refactor-Subagent) als robuster Default-Wahl und bei der Charakterisierung der Workflow×Modell-Interaktion auf novel Katas. Aktive Workflow-Entwicklungs-RQs (`research/workflow-dev/`) sind ausgespart, solange ihre Datenerhebung läuft.

Scope

Der untersuchte Scope ist bewusst eng entlang dreier Achsen: (1) Harness — ausschließlich der **Claude-Code-CLI** (pinned auf `@anthropic-ai/claude-code@2.1.146`, headless ohne HITL); (2) Modelle — ausschließlich **Anthropic-Modelle** (Opus 4.6 und 4.7, Sonnet 4.6, Haiku 4.5 — jeweils mit/ohne Thinking, sowohl Direct-API als auch via Portkey-Gateway); (3) Zielsprache — ausschließlich **TypeScript** mit dem festen `pnpm/tsx/Vitest/ESLint+SonarJS`-Stack pro Run. Diese Eingrenzung eliminiert Tool-, Provider- und Sprach-Variabilität als Confounder; Befunde gelten **für** diesen Stack. Transfer auf andere Agentic-Coding-Tools (Cursor, Aider, Cline, Windsurf), andere Modell-Provider (OpenAI, Google, lokale Modelle), andere Zielsprachen (Python, Go, Java) oder interaktive HITL-Setups ist offen und steht außerhalb dieses Scopes.

AI-Hinweis

Dieser Snapshot wurde mit der `/build-overview`-Skill in **Claude Code** (Anthropic Opus 4.7) erstellt. Datengetriebene Sektionen — RQ-Übersichts-Tabelle, Coverage-Werte, Finding-Listen pro RQ, Reproduzierbarkeits- und Files-Tabelle — werden deterministisch aus `research/{questions,workflow-dev}/*/README,findings.md` via `experiments/generate-snapshot-skeleton.py` generiert. Synthese-Sektionen (Intro, Per-RQ-Paragraphen, Cross-RQ-Synthese, Limitierungen) sind vom LLM gedrafted und human-curated. Die Generierung ist damit vollständig nachvollziehbar.

Hauptbefunde

Vier zentrale Befunde aus den neun Forschungsfragen — Details und Belege in §4, Cross-RQ-Synthese in §5:

- EXACT-Coding wirkt — periodisches TDD-Refactor schlägt Vibe-Coding messbar.** Auf der novel Kata (claim-office) fällt die externe Korrektheit (`verification_pct`) von 1.00 bei TDD-Workflows mit Test-Schreib-Phase auf 0.28 bei reinem Vibe-Coding (v1/v2 ohne Tests). Auf der trainingsbekannten Kata (game-of-life) liefern strikte Workflows mit Refactor-Phase um Faktor ~3 niedrigere Komplexitäts-Spitzen (`cognitive_max`) als alle Non-/Minimal-TDD-Varianten. Praktische Konsequenz: Wenn der Code nicht reiner Wegwerf-Status ist, lohnt sich der Mehraufwand einer TDD-Workflow-Struktur.
- Ein Hybrid-Workflow mit Skill-basiertem Red/Green im geteilten Kontext und isoliertem Refactor-Subagent ist der robuste Default für Opus 4.7.** Diese Architektur ist die einzige, die über beide Code-Qualitäts-Katas (game-of-life, claim-office) in den Top-2 landet, und die einzige mit 0 % Outlier-Rate auf `cognitive_max` (alle 10 Runs $\in [1, 7]$). Der Preis: höchster Token-Verbrauch (Faktor ~17 gegenüber einem End-Refactor nach Vibe-Coding). Für langlebigen Produktiv-Code amortisiert sich der Aufpreis über die Lebensdauer des Codes; für Throwaway-Code bleiben günstigere Varianten attraktiver.
- Example-Mapping ist auf novel Katas der dominante Korrektheits-Hebel — User-Story ≈ Prose.** Auf claim-office hebt Example-Mapping (`verification_pct`) um +48–64 Prozentpunkte gegenüber Prose-Spezifikationen (bei Opus 4.6 und Sonnet 4.6, beide no-thinking), weil konkrete Input/Output-Beispiele die Domänen-Mehrdeutigkeiten auflösen. User-Story-Formulierungen wirken praktisch identisch zu Prose ($\Delta \leq 6$ pp). Auf trainingsbekannten Katas (game-of-life) ist der Effekt null — dort gibt es keine Mehrdeutigkeit, die Beispiele auflösen müssten. Praktische Konsequenz: Beim Schreiben einer Spec für eine novel Domäne sind konkrete Input/Output-Paare die mit Abstand wertvollste Investition — exakt der zweite Schritt im EXACT-Coding-Workflow.
- Refactor-Position trägt mehr als Refactor-Mechanik.** Derselbe spezialisierte Refactor-Subagent (APP-Heuristik + Naming-Eval + Mandatory-Attempt-Klausel) erreicht im periodischen TDD-Zyklus auf claim-office `cognitive_max` 5.0, als End-Refactor nach Vibe-Coding dagegen nur 7.6 — Faktor 1.5 schlechter trotz identischem Prompt. Plausibler Mechanismus: periodisches Refactoring zerlegt Funktionen früh, ein End-Refactor glättet sie nur oberflächlich. Praktische Konsequenz: Wer den Refactor-Subagent als „Aufräumer nach Vibe-Coding“ einsetzt, lässt einen großen Teil seines Werts liegen — der Wert kommt aus der Wiederholung pro Cycle, nicht aus der Mechanik.

1. Forschungsfragen-Übersicht

Forschungsfragen

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-prompt-correctness	Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?	aktiv	24	22/24 (92 %)	126
1.2	RQ-prompt-known-kata	Beeinflusst der Prompt-Stil bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?	aktiv	9	9/9 (100 %)	45
2.1	RQ-model-quality	Wie stark unterscheiden sich die verfügbaren Modelle (Sonnet 4.6, Opus 4.6, Opus 4.7 — jeweils mit/ohne Thinking) in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?	aktiv	6	6/6 (100 %)	25
2.2	RQ-model-novel	Wie unterscheiden sich Opus 4.7 und Opus 4.6 (jeweils no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten?	aktiv	2	2/2 (100 %)	15
3.1	RQ-workflow-model	Hängt die Güte eines TDD-Workflows vom Modell ab — gibt es einen universell besten Workflow, oder tauschen verschiedene Workflows je nach Modell die Plätze?	aktiv	6	6/6 (100 %)	49
4.1	RQ-tdd-quality	Wie wirkt sich die Workflow-Struktur (von oneshot über iterativ bis zu striktem TDD mit Subagents) auf die Code-Qualität aus, und macht die TDD-Striktheit einen Unterschied?	aktiv	16	16/16 (100 %)	101
4.2	RQ-tdd-correctness	Unterscheidet sich die externe Korrektheit (<code>verification_pct</code>) zwischen TDD-Workflow-Varianten auf der neuartigen claim-office-Kata?	aktiv	7	7/7 (100 %)	34
4.3	RQ-context	Welche Form der Kontext-Strukturierung (v4.1 / v5.1 / v6.1 / v7.1) führt zu besserer Code-Qualität?	aktiv	4	4/4 (100 %)	19
5.1	RQ-stability	Wie stabil sind Code-Qualität und TDD-Disziplin pro Workflow über Replikate, und unter welchen Bedingungen ist n=3 ausreichend?	aktiv	6	5/6 (83 %)	59

2. Experiment-Design

2.1 Variablen

Workflow — sechs Generationen (Details: `research/workflow-dev/workflow-overview.md`):

Workflow	Aufbau	TDD-Strenges
v1-oneshot	"Implementiere X."	keine
v2-iterative	"Plane Schritt für Schritt, dann implementiere."	keine
v3-basic-tdd	Inline TDD, kein Skill/Subagent (Self-Reporting)	minimal
v4-exact-subagents	Eigener Subagent pro Phase (Predictor + Red/Green/Refactor), fresh context	strikt, multi-context
v4.1-testlist-scope-fix	v4 mit Test-List-Scope-Patch	strikt, multi-context
v5-exact-single-context	Alle Phasen in einer Konversation, gleiches Phasen-Skript	strikt, single-context
v5.1-testlist-scope-fix	v5 mit Test-List-Scope-Patch (an v4.1 angeglichen)	strikt, single-context
v6-hybrid	Hybrid: inline TDD + nur Refactor als Subagent	strikt, hybrid
v6.1-hybrid-testlist-scope-fix	v6-hybrid mit Test-List-Scope-Patch (aktuelle Default-Basis)	strikt, hybrid
v6.1-no-pep	v6.1 ohne Pep-Talks (RQ-pep-Replikation)	strikt, hybrid
v7-hybrid-green-refactor	Wie v6, aber green und refactor als Subagent	strikt, mehr Isolation
v7.1-hybrid-green-refactor-testlist-scope-fix	v7 mit Test-List-Scope-Patch	strikt, mehr Isolation
v8a-delayed-refactor-agent	Oneshot → nachträgliche Tests → einmaliger End-Refactor-Agent (<code>refactor.md</code> aus v6.5.4)	delayed-refactor
v8b-delayed-refactor-native	Wie v8a, aber nativer Inline-Refactor im v3-Stil, kein Agent	delayed-refactor

Konfiguration: `experiments/workflows/<variant>/` `.claude/agents/` und `.claude/rules/`. Archivierte Varianten (v5.1-minimized, v6.2–v6.6, v6.5.x-Audits) liegen unter `experiments/workflows/_archive/`.

Workflow-Mechanik im Detail. Die sechs Generationen sind nicht nur eine Skala "mehr/weniger TDD", sondern eine systematische Variation der EXACT-Coding-Bausteine (Test-Liste, Red, Green, Refactor) und ihrer Kontext-Architektur:

- **v1-oneshot / v2-iterative — Vibe-Coding-Baselines (kein TDD).** Ein einzelner Agent liest die Anforderungen und schreibt Code in einem Schritt (v1) oder mit explizitem Plan/Checkliste (v2); Tests werden erst nachträglich auf Basis des Example Mappings hinzugefügt. Dient als Messlatte für den Wert von TDD selbst (siehe `experiments/workflows/v1-oneshot/.claude/rules/experiment-mode.md`).
- **v3-basic-tdd — Minimal-TDD ohne Struktur.** Ein einziger Agent mit minimaler Anweisung "use TDD" — keine Phasen-Prompts, keine Subagents. Claude entscheidet selbst, wie es den TDD-Prozess strukturiert. Misst, wie weit eine reine Aufforderung trägt (`v3-basic-tdd/.claude/rules/experiment-mode.md`).
- **v4-exact-subagents / v4.1-testlist-scope-fix — Strict TDD, multi-context.** Jede TDD-Phase läuft als spezialisierter Subagent in **isoliertem Kontext** (Task(subagent_type: "red") etc.): `test-list` → `red` → `green` → `refactor`. Hypothese: isolierte Kontexte erzwingen Disziplin, können aber Zustand zwischen Phasen verlieren. v4.1 ergänzt im `test-list`-Subagent die Pflicht "Cover every spec example" — schließt den dominanten Failure-Mode auf novel Katas (unvollständige Test-Liste) auf Opus 4.7.
- **v5-exact-single-context / v5.1-testlist-scope-fix — Strict TDD, single-context.** Identisches Phasen-Skript wie v4, aber alle Phasen laufen im **gleichen Kontext** als Skill-Calls (Skill(skill: "red") etc.) statt als Subagents. Hypothese: shared context erhält den Zustand, kann aber zu Disziplin-Verlust führen. v5.1 spiegelt v4.1 mit dem identischen Test-List-Scope-Patch.
- **v6-hybrid / v6.1-hybrid-testlist-scope-fix — Hybrid mit isoliertem Refactor.** Red und Green laufen inline als Skills im Shared-Context (wie v5), Refactor läuft als isolierter Subagent (wie v4). Hypothese: kombiniert die Spec-Kohärenz des Single-Context mit der Disziplin-Schärfung der Subagent-Isolation am kritischsten Punkt (Refactor). v6.1 ist die aktuelle Default-Basis und Champion über mehrere RQs. `v6.1-no-pep` testet die Reduktion psychologischer Begründungen in Red/Green.
- **v7-hybrid-green-refactor / v7.1-....-testlist-scope-fix — Hybrid mit isoliertem Green + Refactor.** Zusätzlich zur Refactor-Isolation aus v6 läuft auch Green als isolierter Subagent. Test-Liste und Red bleiben im Shared-Context. Prüft, ob mehr Isolation gleich besser ist (Pareto-dominiert von v6 auf game-of-life: spart Tokens, verliert Qualität und Korrektheit).
- **v8a-delayed-refactor-agent / v8b-delayed-refactor-native — Delayed-Refactor-Kontrolle.** Drei sequentielle Phasen ohne TDD-Cycles: (1) Oneshot-Implementation, (2) nachträgliche Tests gegen `prompt.md` mit Coverage-Pflicht, (3) ein einmaliger End-Refactor. v8a nutzt den `refactor.md`-Subagent aus v6.5.4 (APP + Naming + Mandatory-Attempt), v8b einen nativen Inline-Refactor im v3-Stil ohne Agent. Dient als Kontroll-Achse für die Hypothese "periodisches TDD-Refactor schlägt End-Refactor nach Vibe-Coding".

Tiefere Mechanik-Diskussion und die Reduktions-Genealogie (v6.5.x-Linie, v6.5.4 als Code-Qualitäts-Champion) stehen in `research/workflow-dev/workflow-overview.md` und `workflow-construction.md`. Welche Marker das Parsing der TDD-Metriken treibt, dokumentiert `experiments/workflows/MARKERS.md`.

Modell × Thinking (Lab-Varianten-IDs aus `MODEL_CONFIGS` in `experiments/docker/run-batch.sh`):

Lab-Varianten-ID	API-ID	Thinking	Routing
opus-4-7	claude-opus-4-7	Adaptive	Direct
opus-4-7-no-thinking	claude-opus-4-7	aus	Direct
sonnet-4-6	claude-sonnet-4-6	Extended	Direct
sonnet-4-6-no-thinking	claude-sonnet-4-6	aus	Direct
haiku-4-5	claude-haiku-4-5-20251001	Extended	Direct

Lab-Varianten-ID	API-ID	Thinking	Routing
haiku-4-5-no-thinking	claude-haiku-4-5-20251001	aus	Direct
opus-4-7-portkey	@vertex-eu-global/anthropic.claude-opus-4-7	Adaptive	Portkey
opus-4-7-portkey-no-thinking	@vertex-eu-global/anthropic.claude-opus-4-7	aus	Portkey
opus-4-6-portkey	@vertex-ai/anthropic.claude-opus-4-6	Adaptive	Portkey
opus-4-6-portkey-no-thinking	@vertex-ai/anthropic.claude-opus-4-6	aus	Portkey
sonnet-4-6-portkey	@vertex-ai/anthropic.claude-sonnet-4-6	Extended	Portkey
sonnet-4-6-portkey-no-thinking	@vertex-ai/anthropic.claude-sonnet-4-6	aus	Portkey
haiku-4-5-portkey	@vertex-ai/anthropic.claude-haiku-4-5@20251001	Extended	Portkey
haiku-4-5-portkey-no-thinking	@vertex-ai/anthropic.claude-haiku-4-5@20251001	aus	Portkey

Direct- und Portkey-Routings desselben Modells sind getrennte Varianten und werden nur per expliziter `controls.model: {any: [...]}`-Klausel pro RQ als gemeinsame Zelle gewertet.

Kata × Prompt-Stil (aktive Katas in `experiments/katas/`):

Kata-Basis	Prompt-Stile	Verifikations-Suite	Hinweis
game-of-life	prose, example-mapping, user-story	nein	Code-Qualität, groß (~40 LoC), vitest-basiert
game-of-life-cli	prose, example-mapping, user-story	ja	CLI-Variante mit externer Akzeptanz-Suite
mars-rover	prose, example-mapping, user-story	nein	mittel (~30 LoC), vitest-basiert
claim-office	prose, example-mapping, user-story	ja	Korrektheit, novel Versicherungs-Domäne (HPSMV/MHPCO), 15 Szenarien
claim-office-lite	prose, example-mapping, user-story	ja	Reduzierte claim-office-Variante (10 Szenarien) für Code-Qualitäts-Research

Prompt-Stile: - **prose**: Beschreibung der Regeln in Prosa, keine Test-Beispiele. - **example-mapping**: Regel + 1–3 konkrete Input/Output-Beispiele pro Regel. - **user-story**: "Als X möchte ich Y, damit Z" — Beschreibung ohne Beispiele.

2.2 Workflow → Prompt-Mapping

Aus methodischer Symmetrie (siehe `Top-README.md`, Abschnitt 'Methodology constraints'):

Workflow	erlaubte Prompt-Stile	Begründung
v1, v2	nur prose	Test-Beispiele in example-mapping wären für Non-TDD-Workflows ein verstecktes Test-Geschenk → unfair gegenüber den TDD-Workflows.
v3, v4(.1), v5(.1), v6(.1), v7(.1), v8a/b	alle drei	Beispiele dienen als natürliche Test-Cases — für TDD-/Refactor-Workflows ist das das Idealbild der Aufgabe.

3. Methodik

Pipeline gegenüber dem v2-Snapshot (2026-05-04) im Wesentlichen unverändert; einzige Drift: Container-Pin auf `@anthropic-ai/claude-code@2.1.146` (vorher 2.1.107). `analyze-run.sh` und `aggregate-by-query.py` sind in Kern-Logik stabil; neue Felder (`mccabe_*`, `cognitive_*`, `median_loc_per_function`, `verification_pct`, `mutation_score`) wurden additiv ergänzt.

3.1 Run-Pipeline

- 1. Container-Image `docker-batch` (Node 22 slim, `@anthropic-ai/claude-code@2.1.146` gepinnt) wird gestartet.
- 2. `Run-Dir runs/<timestamp>_<kata>_<workflow>_<model>/` wird angelegt; Workflow-Konfig (`.claude/agents/`, `.claude/rules/`) und Kata-Prompt (`prompt.md`) hinein kopiert.
- 3. `pnpm-Workspace` mit TypeScript, Vitest, ESLint+SonarJS aufgesetzt.
- 4. `claude --print "$(< prompt.md)"` läuft headless, ohne HITL.
- 5. `analyze-run.sh` schreibt `metrics.json` und `analysis-report.md`.
- 6. `aggregate-by-query.py <RQ>/` baut `runs.csv` und `summary.md` pro RQ.

3.2 Erfasste Metriken

Verbindliche Termini (Spalte "Term") sind im Top-README.md definiert — alternative Synonyme sind verboten, weil sie kollidieren oder mehrdeutig sind. Volle Metrik-Tabelle inklusive externer Referenzen (Stryker, SonarJS, McCabe-Paper etc.) im README Abschnitt "Metrics".

Korrektheit

Metrik	Term	Was misst es	Richtung
tests_passing	Korrektheit (innen)	Boolean: laufen die vom Agenten geschriebenen Vitest-Tests am Ende des Runs grün?	true = besser
verification_pct	Korrektheit (außen)	Anteil bestandener Verifikations-Szenarien aus einer externen Acceptance-Suite, die der Agent nie zu sehen bekommt (0.0–1.0). Nur für CLI-Katas mit <basename>-verification/ -Verzeichnis.	höher = besser

Effizienz

Metrik	Term	Was misst es	Richtung
duration_seconds	—	Wallclock-Sekunden des claude --print -Runs inkl. aller Subagent-Spawns	kleiner = besser
total_tokens	—	Summe aller Tokens (Input + Output + Cache) über alle Subagent-Spawns hinweg	kleiner = besser
context_utilization_pct	—	Finale Context-Window-Auslastung im Main-Context, in Prozent	informativ

Code-Mass & Umfang

Metrik	Term	Was misst es	Richtung
code_mass	Code-Mass (APP)	Gewichtete Summe der Produktiv-Code-Konstrukte (Konstanten, Invocations, Conditionals, Loops, Assignments — gestaffelte Gewichte nach Komplexität) gemäß <i>Absolute Priority Premise</i> (Micah Martin). Vergleicht Implementationen objektiver als reine LoC.	kleiner = besser
cc_loc	Produktiv-LoC	Produktiv-LoC ohne Tests, aus dem Clean-Code-Reporter	kleiner = besser (bei gleicher Korrektheit)
test_lines	Test-LoC	Anzahl Zeilen Test-Code (Vitest)	informativ
tests_total	—	Anzahl vom Agenten geschriebener Tests	informativ

Code-Qualität (ESLint + SonarJS)

Metrik	Term	Was misst es	Richtung
cc_longest_function	Spitzen-Komplexität	Längste Funktion in Zeilen — Proxy für die schlechteste Stelle im Code	kleiner = besser
cc_avg_loc_per_function	—	Mittlere Funktionsgröße in Zeilen	kleiner = besser
cc_median_loc_per_function	—	Median-Funktionsgröße (robust gegen einzelne lange Outlier)	kleiner = besser
cc_functions	—	Anzahl Funktionen	informativ
mccabe_max / mccabe_avg / mccabe_high_count	—	McCabe Cyclomatic Complexity pro Funktion: Maximum, Mittel, Anzahl über Schwellwert. Klassische Verzweigungs-Metrik.	kleiner = besser
cognitive_max / cognitive_avg / cognitive_high_count	—	SonarSource Cognitive Complexity pro Funktion: gewichtet Nesting und Control-Flow-Breaks stärker als McCabe, näher an menschlich wahrgenommener Komplexität. Diagnostisch tragende Hauptmetrik dieser Studie.	kleiner = besser
smell_total	Smell-Summe	Aggregierte Anzahl ESLint+SonarJS-Verstöße über alle Regeln	kleiner = besser
smell_complexity	—	Subset von smell_total: cognitive-complexity, max-depth, max-lines-per-function, max-params, no-nested-switch	kleiner = besser
smell_magic_numbers	—	Subset: ESLint no-magic-numbers - Verstöße	kleiner = besser
smell_duplication	—	Subset: SonarJS no-duplicate-string und verwandte Duplikations-Regeln	kleiner = besser
smell_code_quality	—	Subset: SonarJS no-collapsible-if, no-redundant-jump etc., plus ESLint no-unreachable	kleiner = besser

Metrik	Term	Was misst es	Richtung
coverage_statements_pct	—	Statement-Coverage der vom Agenten geschriebenen Tests (in %)	höher = besser
coverage_branches_pct	—	Branch-Coverage der vom Agenten geschriebenen Tests (in %)	höher = besser

Test-Stärke

Metrik	Term	Was misst es	Richtung
mutation_score	Mutation-Score	Anteil der Stryker-Mutanten, die von der Test-Suite des Agenten gekillt werden (0.0–1.0): $(\text{Killed} + \text{Timeout}) / (\text{Killed} + \text{Survived} + \text{Timeout} + \text{NoCoverage})$. Hidden Metric — kommt in keinem Workflow-Prompt vor, daher Goodhart-resistent. Opt-in per RQ, nur für <code>tests_passing = true</code> .	höher = besser

TDD-Disziplin (aus `transcript.jsonl` + `transcript-subagents/`; getrieben von vier Markern in `experiments/workflows/MARKERS.md` — fehlt ein Marker, fällt die zugehörige Metrik still auf null)

Metrik	Term	Was misst es	Richtung
cycle_count	—	Anzahl Red-Green-Refactor-Zyklen pro Run	informativ (höher = feiner zerlegt)
refactorings_applied	—	Anzahl explizit angewandter Refactoring-Schritte	höher = besser (bei TDD-Workflows)
predictions_correct / predictions_total	—	Red-Phase-Vorhersagen über Compile-/Runtime-Failure: korrekt vs. gesamt. Tiefe des Code-Verständnisses des Agenten. Pro Cycle 1–2 Predictions je nach Workflow.	Quote höher = besser
tests_passed_immediately	—	Anzahl Tests, die in der Red-Phase bereits grün waren — Indikator für Over-Implementation in vorherigen Green-Phasen	kleiner = besser
avg_red_seconds / avg_green_seconds / avg_refactor_seconds	—	Mittlere Phasendauer pro Cycle	informativ

3.3 Bewertungsgrundsätze

- **Korrektheit zuerst:** ein Run mit `tests_passing = false` zählt nicht als gleichwertige Lösung.
- **Pro Kata aggregieren:** Workflow×Modell-Tabellen werden ausschließlich pro Kata gebildet.
- **Effekt-Schwelle:** Bei $n=1$ pro Zelle gelten nur Differenzen mit Faktor ≥ 2 oder klar getrennten σ -Bändern als belastbar.
- **Goodhart-Hygiene:** Metriken, die in einem Workflow-Prompt namentlich genannt werden, sind für diesen Workflow Compliance-Signale, keine unabhängigen Outcomes.

4. Ergebnisse

Forschungsfragen

1.1 RQ-prompt-correctness — Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?

Datenbasis: 126 Runs · Coverage: 22/24 Zellen (92 %) bei `min_replicates=5`.

Übersicht — `verification_pct` (Korrektheit außen) nach Modell × Prompt-Stil × Thinking (höher = besser; 🏆 = bester Stil pro Zeile):

Modell	Modus	prose	example-mapping	user-story
opus-4-7	-thinking	—	1.00 🏆 (n=3)	—
opus-4-6	-thinking	0.23	0.87 🏆	0.23
opus-4-6	+thinking	0.15	0.77 🏆	0.25
sonnet-4-6	-thinking	0.23	0.71 🏆	0.17
sonnet-4-6	+thinking	0.21	0.35 🏆	0.19
haiku-4-5	-thinking	0.00	0.00	0.00
haiku-4-5	+thinking	0.00	0.00	0.01

Befunde:

- **F-prompt-correctness.1** — Schwache Modelle scheitern unabhängig vom Prompt-Stil
- **F-prompt-correctness.2** — Example-Mapping hebt Korrektheit massiv

- **F-prompt-correctness.3** — Thinking schadet bei Example-Mapping (Sonnet > Opus)
- **F-prompt-correctness.4** — User-Story ≈ Prose, keine messbare Wirkung auf Korrektheit
- **F-prompt-correctness.5** — Streuung bei Example-Mapping ist modellabhängig

Auf `claim-office` differenziert Example-Mapping `verification_pct` bei Opus 4.6 und Sonnet 4.6 (no-thinking) um +48–64 Prozentpunkte gegenüber Prose, weil konkrete Input/Output-Beispiele die HPSMV-Mehrdeutigkeiten auflösen. User-Story wirkt fast identisch zu Prose ($\Delta \leq 6$ pp). Haiku 4.5 scheitert stilunabhängig (`verification_pct` ≈ 0) — Reasoning-Kapazität fehlt für die Generalisierung. Thinking schadet bei Sonnet × Example-Mapping deutlich (–36 pp), bei Opus 4.6 nur leicht (–10 pp); Transcript-Analyse zeigt Sonnet konstruiert mit Thinking alternative Lesarten der Spec. Coverage 22/24 Zellen — zwei Opus-4-7-Zellen ohne Replikate. Details: [research/questions/1.1-prompt-style-correctness/findings.md](#).

1.2 RQ-prompt-known-kata — Beeinflusst der Prompt-Stil bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?

Datenbasis: 45 Runs · Coverage: 9/9 Zellen (100 %) bei `min_replicates=5`.

Übersicht — `verification_pct` nach Prompt-Stil × Modell (höher = besser; 🏆 = bester Stil pro Zeile):

Modell	prose	user-story	example-mapping
opus-4-6-portkey-no-thinking	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)
sonnet-4-6-portkey-no-thinking	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)
haiku-4-5-portkey-no-thinking	0.24 ($\sigma=0.43$)	0.00 ($\sigma=0$)	0.63 🏆 ($\sigma=0.51$)

Befunde:

- **F-prompt-known-kata.1** — Opus und Sonnet liefern stilunabhängig perfekte Korrektheit
- **F-prompt-known-kata.2** — Haiku scheitert kapazitätsbedingt, nicht stilbedingt
- **F-prompt-known-kata.3** — H1 bestätigt: Prompt-Stil differenziert bei starken Modellen nicht
- **F-prompt-known-kata.4** — H4 bestätigt: Mehrdeutigkeits-Mechanismus greift nicht bei trainingsbekannter Kata
- **F-prompt-known-kata.5** — H2 kann nicht bewertet werden: Code-Qualität nur bei funktionierenden Runs vergleichbar
- **F-prompt-known-kata.6** — RQ-prompt-correctness-Prognose bestätigt: Prompt-Stil differenziert nicht auf trainingsbekannter Kata
- **F-prompt-known-kata.7** — Verification-Adapter eliminiert Interface-Artefakte

Auf trainingsbekannter Game-of-Life-Kata erreichen Opus 4.6 und Sonnet 4.6 (no-thinking) über alle drei Prompt-Stile `verification_pct` = 1.00 (30/30 Runs, $\sigma=0$) — der Prompt-Stil ist auf bekannter Domäne kein Korrektheits-Hebel mehr. Haiku zeigt bimodales "Sofort-Aufgeber-vs-Durchläufer"-Verhalten: Example-Mapping aktiviert in 4/5 Runs den Arbeitsmodus (mean 0.63), User-Story in 0/5 Runs (mean 0.00). Code-Qualität variiert bei Opus/Sonnet nicht systematisch zwischen Stilen. Konsequenz: Code-Qualitäts-RQs auf Game-of-Life dürfen Prompt-Stil als Control fixieren. Coverage 9/9 Zellen. Details: [research/questions/1.2-prompt-style-known-kata/findings.md](#).

2.1 RQ-model-quality — Wie stark unterscheiden sich die verfügbaren Modelle in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?

Datenbasis: 25 Runs · Coverage: 6/6 Zellen (100 %) bei `min_replicates=3`.

Übersicht — Code-Qualität nach Modell auf v4 × `game-of-life-example-mapping` (kleiner = besser, außer `verification_pct`; 🏆 = bester Wert pro Spalte):

Modell	code_mass	smell_total	mccabe_max	cognitive_max	cc_longest_function	verification_pct	n
opus-4-7	159.00 🏆	2.33 🏆	3.33 🏆	3.00	7.00 🏆	1.00 🏆	3
opus-4-7-no-thinking	167.67	2.50	4.00	2.83 🏆	9.33	1.00 🏆	6
opus-4-6-portkey	173.00	4.33	6.67	12.00	19.33	1.00 🏆	3
opus-4-6-portkey-no-thinking	175.67	4.33	7.67	13.00	18.67	1.00 🏆	3
sonnet-4-6	178.00	5.67	6.33	11.00	21.67	1.00 🏆	3
sonnet-4-6-no-thinking	166.67	3.33	6.00	5.00	15.00	0.73	3

Befunde:

- **F-model-quality.1** — Korrektheit (innen + außen) auf v4 ist nahezu modellunabhängig perfekt
- **F-model-quality.2** — Modell-Ranking: Opus-4.7 deutlich vor Sonnet-4.6 und Opus-4.6; Sonnet jetzt vor Opus-4.6
- **F-model-quality.3** — Thinking wirkt nicht uniform; Opus-4.6 + Thinking ohne Vorteil, Sonnet + Thinking sogar negativ auf `cognitive_max`
- **F-model-quality.4** — Token-Kosten und Wallclock nivellieren sich auf v4 weitgehend
- **F-model-quality.5** — Vertrags-Konformität unter explizitem API-Vertrag fast vollständig erreicht; ein Sonnet-Ausreißer redefiniert `Cell` als Objekt

Opus 4.7 (no-thinking) liefert auf v4 × `game-of-life-example-mapping` die niedrigsten Komplexitäts-Spitzen (`cognitive_max` 2.83, `mccabe_max` 4.0, `cc_longest_function` 9.33) und liegt damit klar vor Sonnet 4.6 (no-thinking) und Opus 4.6; Sonnet überholt dabei Opus 4.6, was die naive Tier-Intuition umkehrt. Korrektheit ist mit explizitem API-Vertrag in 5/6 Zellen perfekt; ein Sonnet-no-thinking-Ausreißer redefiniert `Cell` als Objekt (`verification_pct` 0.73). Thinking wirkt nicht uniform: bei Opus neutral, bei Sonnet × `cognitive_max` deutlich negativ (5.00 → 11.00). Token-Spread zwischen Modellen nur 1.75×. Caveat: n=3 pro Zelle (außer opus-4-7-no-thinking n=6), single workflow, single kata. Details: [research/questions/2.1-model-effect-code-quality/findings.md](#).

2.2 RQ-model-novel — Wie unterscheiden sich Opus 4.7 und Opus 4.6 (no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten?

Datenbasis: 15 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=5.

Übersicht — **verification_pct** auf **claim-office-example-mapping** × **v4-exact-subagents** (höher = besser; 🏆 = bester Wert):

Modell	n	verification_pct	σ
opus-4-6-portkey-no-thinking	5	0.93 🏆	0.08
opus-4-7-no-thinking	10	0.67	0.36

Befunde:

- **F-model-novel.1** — opus-4-6 schlägt opus-4-7 auf v4 × claim-office
- **F-model-novel.2** — Workflow×Modell-Interaktion ist der dominierende Effekt
- **F-model-novel.3** — Korrektheit differenziert stärker als Code-Qualität
- **F-model-novel.4** — Präziserer Mechanismus auf opus-4-7: Test-Listen-Vollständigkeit, nicht Subagent-Isolation

Auf claim-office-example-mapping × v4-exact-subagents schlägt Opus 4.6 (no-thinking, Portkey) das neuere Opus 4.7 (no-thinking) deutlich (verification_pct 0.93 vs. 0.67, σ 0.08 vs. 0.36). Der Mechanismus ist eine Workflow×Modell-Interaktion: auf v6-hybrid kehrt sich das Ranking um (Opus 4.7: 1.00, Opus 4.6: 0.68); v5 ist modell-unabhängig konstant (0.87). Die verfeinerte Mechanik aus RQ-tdd-correctness: der dominante 4.7-Failure-Mode auf v4 ist eine unvollständige Test-Liste — v4.1 mit Spec-Coverage-Pflicht schließt die Lücke (0.96 vs. 0.67). Caveat: 4.6 läuft via Portkey, 4.7 via Direct API — Routing-Confound möglich. Details: research/questions/2.2-model-effect-novel-kata/findings.md.

3.1 RQ-workflow-model — Hängt die Güte eines TDD-Workflows vom Modell ab?

Datenbasis: 49 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

Übersicht — **verification_pct** auf **claim-office-example-mapping** nach **Workflow** × **Modell** (höher = besser; 🏆 je Modell-Spalte — Sieger wechselt modell-abhängig):

Workflow	opus-4-7 (n)	opus-4-6 (n)
v4-exact-subagents	0.67 (10)	0.93 (5) 🏆
v5-exact-single-context	0.87 (10)	0.87 (5)
v6-hybrid	1.00 (5) 🏆	0.68 (15)

Befunde:

- **F-workflow-model.1** — v4 und v6 tauschen je nach Modell die Plätze
- **F-workflow-model.2** — Mechanismus: Orchestrierungs-Delegation vs. expliziter Subagent-Prompt

v4 und v6 sind auf claim-office-example-mapping modell-abhängig komplementär: v6-hybrid ist Opus-4-7-Optimum (verification_pct 1.00) und auf Opus 4.6 instabil (0.68); v4-exact-subagents ist auf Opus 4.6 stabil (0.93) und auf Opus 4.7 bimodal (0.67). v5-exact-single-context ist modell-unabhängig konstant (0.87). Mechanismus: v6-hybrid delegiert die Orchestrierung an das Modell (Skill-Invocation im Shared-Context) — Opus 4.7 beherrscht das, Opus 4.6 verliert in ~40 % der Runs die Claim-Hälfte der Spec (implementiert nur Quote). Es gibt damit keinen universell besten Workflow auf dieser Achse; die Praxis-Empfehlung steht in research/workflow-dev/model-recommendation-matrix.md. Details: research/questions/3.1-workflow-model-interaction/findings.md.

4.1 RQ-tdd-quality — Wie wirkt sich die Workflow-Struktur auf die Code-Qualität aus, und macht die TDD-Striktheit einen Unterschied?

Datenbasis: 101 Runs · Coverage: 16/16 Zellen (100 %) bei min_replicates=5.

Übersicht — **Code-Qualität pro Workflow, getrennt nach Kata** (kleiner = besser; 🏆 = bester Wert pro Spalte; ⚠️ = bimodal):

Kata: **game-of-life** (opus-4-7-no-thinking, prose für v1/v2, example-mapping sonst)

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v1-oneshot	10	18.8	12.8	31.7	4.8	33.6	155.0
v2-iterative	10	16.2	11.6	32.1	4.1	32.5	157.8
v3-basic-tdd	10	21.8	13.7	32.5	6.0	31.9	165.6
v4.1-testlist-scope-fix	5	6.4 🏆	5.0 🏆	16.4	2.4	32.0	156.6
v5.1-testlist-scope-fix	5	17.6	10.2	20.8	4.8	26.6 🏆	154.0
v6.1-hybrid-...	5	6.8	5.2	13.4 🏆	2.2 🏆	31.0	159.8
v8a-delayed-refactor-agent	5	11.0	6.8	16.6	3.4	33.0	147.2 🏆
v8b-delayed-refactor-native	5	8.4	6.0	13.8	3.2	33.0	158.2

Kata: **claim-office** (opus-4-7-no-thinking, prose für v1/v2, example-mapping sonst)

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v1-oneshot	5	12.2	8.4	40.4	11.6	269.4	835.4

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v2-iterative	5	11.4	8.4	41.4	15.8	268.6	851.0
v3-basic-tdd	5	19.8	15.4	51.6	16.8	317.4	992.4
v4.1-testlist-scope-fix	5	26.8 🚩	16.0 🚩	40.8	13.2	156.8 🏆	621.6 🏆
v5.1-testlist-scope-fix	6	14.8	10.2	32.7	6.8	167.2	692.7
v6.1-hybrid-...	5	5.0 🏆	5.2 🏆	18.4 🏆	1.6	198.4	878.6
v8a-delayed-refactor-agent	5	7.6	5.8	25.6	7.2	228.0	777.4
v8b-delayed-refactor-native	5	8.4	7.0	25.0	1.0 🏆	259.2	812.0

Korrektheit (`verification_pct`): auf game-of-life alle 8 Workflows = 1.00; auf claim-office variiert von 0.28 (v1/v2) bis 1.00 (v3, v5.1, v6.1) — siehe F-tdd-quality.4 und F-tdd-quality.8.

Befunde:

- **F-tdd-quality.1** — Strikte phasen-strukturierte Workflows mit Refactor-Phase senken die Komplexitäts-Spitzen drastisch
- **F-tdd-quality.2** — Minimal-TDD (v3) bringt auf game-of-life keinen Komplexitäts-Vorteil gegenüber Non-TDD (v1/v2)
- **F-tdd-quality.3** — Single-Context (v5.1) verliert den Komplexitäts-Vorteil der phasen-isolierten Subagents (v4.1) — aber nur auf game-of-life
- **F-tdd-quality.4** — Korrektheit ist workflow-abhängig auf novel Kata; v1/v2 Vibe-Coding kollabiert auf claim-office
- **F-tdd-quality.5** — Kostenspanne zwischen Workflows ist eine Größenordnung; strikte Workflows sind 5–50× teurer; Kata-Komplexität skaliert linear
- **F-tdd-quality.6** — Vibe + End-Refactoring erreicht Volumen-Niveau der strikten TDD-Workflows zu Non-TDD-Kosten; Verzweigungs-Komplexität bleibt schwächer
- **F-tdd-quality.7** — APP-Refactor-Subagent als End-Refactor schadet auf game-of-life; auf claim-office gemischt
- **F-tdd-quality.8** — Test-Schreib-Phase rettet Korrektheit auf novel Kata; reines Vibe-Coding scheitert
- **F-tdd-quality.9** — v6.1-Hybrid ist der robusteste TDD-Workflow über beide Katas; v4.1 ist kata-instabil

Strikte phasen-strukturierte Workflows mit Refactor-Phase (v4.1, v6.1) senken `cognitive_max` und `mccabe_max` auf game-of-life auf ~½ der v1/v2/v3-Werte. v6.1-hybrid ist der einzige Workflow, der auf beiden Katas (game-of-life, claim-office) in den Top-2 landet — auf claim-office sogar Champion bei `cognitive_max` (5.0) und `mccabe_max` (5.2). v4.1 dagegen kollabiert auf claim-office (bimodal, $\sigma=24$, $\max=68$) — der frische Subagent-Kontext verliert auf langen Test-Listen die Kohärenz. v1/v2 ohne Tests brechen auf claim-office auf `verification_pct` = 0.28 ein; jede Test-Schreib-Phase rettet ≥ 0.96 . Kostenspanne zwischen v6.1 (33 M Tokens) und v8a/v8b (1.9–3.1 M) liegt bei Faktor 5–17×. Details: [research/questions/4.1-tdd-effect-code-quality/findings.md](#).

4.2 RQ-tdd-correctness — Unterscheidet sich die externe Korrektheit zwischen TDD-Workflow-Varianten auf claim-office?

Datenbasis: 34 Runs · Coverage: 7/7 Zellen (100 %) bei `min_replicates=3`.

Übersicht — Korrektheit pro Workflow auf claim-office-example-mapping × opus-4-7-no-thinking (höher = besser, 🏆 = bester Wert pro Spalte):

Workflow	n	verification_pct (mean ± std)	verification_passed / 15 (min – max)	tests_passing
v3-basic-tdd	5	1.00 ± 0 🏆	15 – 15	100 % 🏆
v4.1-testlist-scope-fix	5	0.96 ± 0.09	12 – 15	100 % 🏆
v5.1-testlist-scope-fix	6	1.00 ± 0 🏆	15 – 15	100 % 🏆
v6.1-hybrid-...	3	1.00 ± 0 🏆	15 – 15	100 % 🏆
v7.1-hybrid-green-refactor-...	3	0.98 ± 0.04	14 – 15	100 % 🏆

`completed_within_budget` in allen Zellen 100 %.

Befunde:

- **F-tdd-correctness.1** — Drei von fünf TDD-Workflows lösen claim-office perfekt; v4.1 und v7.1 verlieren vereinzelt Szenarien
- **F-tdd-correctness.2** — v4.1 erreicht Korrektheit nur über drastisch höheren Aufwand pro Zyklus
- **F-tdd-correctness.3** — Predictions-Rate-Vergleich ist verzerrt durch ungleiche Vorhersage-Basis
- **F-tdd-correctness.4** — Wallclock-Spanne ist 10×, Token-Spanne 9×; keine Korrektheits-Korrelation

Auf claim-office-example-mapping × Opus 4.7 (no-thinking) lösen v3, v5.1 und v6.1 die externe Verifikations-Suite in jedem Run perfekt (15/15). v4.1 und v7.1 verlieren je einen Run (0.96 / 0.98) — auffällig beide mit isoliertem Green-Subagent. v4.1 erkaufte seine Korrektheit über 44.6 Cycles und 14 M Tokens (vs. v3: 3.8 Cycles, 3.3 M Tokens, ebenfalls 100 %) und liegt damit beim schlechtesten Cost-Quality-Tradeoff. Wallclock-Spanne 10×, Token-Spanne 9×, ohne Korrektheits-Korrelation. Strukturierte Workflows rechtfertigen sich auf claim-office nicht durch Korrektheit — ihr Wert liegt in Code-Qualität (vgl. RQ-context). Details: [research/questions/4.2-tdd-effect-correctness/findings.md](#).

4.3 RQ-context — Welche Form der Kontext-Strukturierung führt zu besserer Code-Qualität?

Datenbasis: 19 Runs · Coverage: 4/4 Zellen (100 %) bei `min_replicates=3`.

Übersicht — Code-Qualität, Korrektheit und Kosten auf claim-office-example-mapping × opus-4-7-no-thinking (Komplexität/Smells/Kosten kleiner = besser, `verification_pct` höher = besser; 🏆 = bester Wert pro Spalte, Ties bei Spread < 1 σ):

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	code_mass	cc_loc	verification_pct	duration_seconds	total_tokens
v4.1 (alle isoliert)	5	26.8 ± 24.1	16.0 ± 9.0	40.8 ± 27.1	13.2 ± 7.5	621.6 ± 65.6 🏆	156.8 ± 38.0 🏆	0.96 ± 0.09	3 229 ± 920	14.10 M ± 2.99 🏆
v5.1 (alle geteilt)	6	14.8 ± 4.2	10.2 ± 2.6	32.7 ± 10.2	6.8 ± 7.6	692.7 ± 78.8	167.2 ± 27.9	1.00 ± 0 🏆	641 ± 122 🏆	18.73 M ± 5.35
v6.1 (Refactor isoliert)	3	4.3 ± 1.5 🏆	5.0 ± 1.7 🏆	16.7 ± 6.7 🏆	1.3 ± 1.2 🏆	920.7 ± 55.2	184.3 ± 4.9	1.00 ± 0 🏆	1 424 ± 781	30.16 M ± 18.56
v7.1 (Green + Refactor isoliert)	3	5.0 ± 1.0 🏆	4.67 ± 0.58 🏆	19.3 ± 2.5 🏆	2.3 ± 2.3 🏆	801 ± 3.6	187.3 ± 29.2	0.98 ± 0.04	1 970 ± 715	26.11 M ± 6.20

tests_passing und completed_within_budget in allen vier Zellen 100 %.

Befunde:

- **F-context.1** — Refactor-Subagent liefert den Komplexitäts-Vorteil; zusätzliche Green-Isolation ändert das Bild nicht
- **F-context.2** — Refactor-Subagent verteilt Funktionalität auf mehr Bausteine; Green-Isolation bremst den Mehr-Code-Effekt
- **F-context.3** — Korrektheit unterscheidet die Architekturen nicht
- **F-context.4** — Vier sehr unterschiedliche Kosten-Profile
- **F-context.5** — Zwei Hybrid-Positionen mit ähnlicher Code-Qualität, unterschiedlichem Kosten-Profil

Auf claim-office-example-mapping liefert der **isolierte Refactor-Subagent** den Komplexitäts-Vorteil: v6.1 und v7.1 erreichen praktisch identische Spitzen (cognitive_max 4.3 / 5.0, mccabe_max 5.0 / 4.67, cc_longest_function 16.7 / 19.3). Zusätzliche Green-Isolation (v7.1) bringt keinen Hub mehr. v4.1 (alle Phasen isoliert) ist auf dieser Kata schlechter als v5.1 (alle geteilt) in allen vier Spitzen-Metriken — die paarweise Hypothese "Isolation > Shared-Context" wird falsifiziert. Korrektheit unterscheidet die Architekturen nicht (alle ≥ 0.96). v5.1 ist mit 11 min Wallclock 5× schneller als v4.1 (54 min); v7.1 hat die kleinste Streuung über alle Metriken. Cross-Kata-Replikation auf mars-rover offen. Details: [research/questions/4.3-tdd-context-engineering/findings.md](#).

5.1 RQ-stability — Wie stabil sind Code-Qualität und TDD-Disziplin pro Workflow über Replikate, und unter welchen Bedingungen ist n=3 ausreichend?

Datenbasis: 59 Runs · Coverage: 5/6 Zellen (83 %) bei min_replicates=10.

Übersicht — Code-Qualität nach Workflow (n=10) auf game-of-life × opus-4-7-no-thinking (kleiner = besser; bester Wert pro Spalte fett):

Workflow	code_mass	smell_total	mccabe_max	cognitive_max	cc_longest_function	n
v1-oneshot (prose)	155.00	4.80	12.80	18.80	31.70	10
v2-iterative (prose)	157.80	4.10	11.60	16.20	32.10	10
v3-basic-tdd (EM)	165.60	6.00	13.70	21.80	32.50	10
v4-exact-subagents (EM)	166.60	2.60	4.50	4.40	8.10	10
v5-exact-single-context (EM)	152.60	4.10	8.90	14.50	17.40	10
v6-hybrid (EM)	158.60	2.20	4.50	5.20	13.10	10

Befunde:

- **F-stability.1** — RQ-tdd-quality-Hauptbefund (v4 dominiert Code-Komplexität, v3 ist Schlusslicht) repliziert bei n=10 mit gleichem Vorzeichen
- **F-stability.2** — Workflow-Stabilität ist nicht uniform; v4 hat 10 %-Outlier-Rate trotz tiefem typischen Wert; v5 ist breitesten Workflow
- **F-stability.3** — Bei n=3 ist die volle Workflow-Rangordnung nur in ~25–60 % der Fälle korrekt; v4 als "Bester" ist robuster
- **F-stability.4** — Korrektheit bleibt bei n=10 modell-/workflow-unabhängig 100 %
- **F-stability.5** — Token-Verbrauch zeigt extrem hohe Streuung bei v4 und v5
- **F-stability.6** — TDD-Disziplin bildet workflow-charakteristische Banden
- **F-stability.7** — Test-Stärke (mutation_score) hat eigenes Stabilitätsprofil; v6-hybrid ist stabilster Workflow, v4 instabiler

Bei n=10 auf game-of-life × opus-4-7-no-thinking repliziert das RQ-tdd-quality-Ranking vollständig: v4 dominiert Code-Komplexität (cognitive_max 4.4), v3 ist Schlusslicht (21.8). Workflow-Stabilität ist nicht uniform: v6-hybrid ist der einzige Workflow mit 0 % Outlier-Rate (alle 10 Runs cognitive_max ∈ [1, 7]); v4 hat trotz niedrigem Median (3) eine 10 %-Outlier-Rate; v5 ist breitbandig (σ 4.59). Bei n=3 ist die volle 5-Workflow-Rangordnung auf den Code-Qualitäts-Metriken nur in ~25–60 % der Subsamples korrekt — Mittelfeld-Vergleiche brauchen n=10+. mutation_score zeigt v6 als stabilsten Workflow (σ 0.005, min 0.940), v4 als instabilsten (σ 0.080, min 0.735). Coverage 5/6 Zellen (50/60 Runs). Details: [research/questions/5.1-workflow-stability/findings.md](#).

5. Cross-RQ-Synthese

1. **Modell-Vergleiche sind nur innerhalb fixierter Workflow- und Kata-Achsen aussagekräftig.** RQ-model-quality (auf v4 × game-of-life) und RQ-model-novel (auf v4/v5/v6 × claim-office) liefern teils gegensätzliche Modell-Rankings: Opus 4.7 schlägt Opus 4.6 auf Game-of-Life-Code-Qualität klar, Opus 4.6 schlägt Opus 4.7 auf claim-office × v4-Korrektheit ebenso klar (0.93 vs. 0.67). RQ-workflow-model schärft das: v4 und v6 tauschen je nach Modell die Plätze. Aussagen wie "Opus 4.7 ist besser" sind ohne Workflow×Kata-Kontext nicht generalisierbar.
2. **Refactor-Position ist wichtiger als Refactor-Mechanik.** RQ-tdd-quality und RQ-context zeigen unabhängig voneinander: was v6.1 von v3/v5.1 trennt, ist die Existenz eines **isolierten Refactor-Subagents pro Cycle**, nicht der Inhalt des Refactor-Prompts. v8a mit identischem refactor.md-Subagent als

End-Refactor liefert deutlich schlechtere `cognitive_max` (7.6 vs. 5.0 auf claim-office) — die Periodizität trägt; das Subagent-Format als Einmal-Schuss reicht nicht.

- 3. **Test-Schreib-Phase ist der Korrektheits-Hebel auf novel Katas, nicht TDD-Striktheit.** RQ-tdd-quality (v8a/v8b mit nachträglichen Tests erreichen `verification_pct` 0.97; v3 mit minimaler TDD-Aufforderung erreicht 1.00) und RQ-tdd-correctness (v3 = v5.1 = v6.1 = 1.00) zeigen: für Korrektheit zählt ob Tests gegen die Spec geschrieben werden, nicht wann im Cycle. Vibe-Coding ohne Tests (v1/v2) kollabiert dagegen auf 0.28.
- 4. **n=3 trägt nur für große Effekte; Mittelfeld-Vergleiche brauchen n=10+.** RQ-stability quantifiziert, was in RQ-tdd-quality, RQ-model-quality und RQ-context als Caveat steht: bei n=3 ist die volle Workflow-Rangordnung auf den Code-Qualitäts-Metriken nur in 25–60 % der Subsamples korrekt. "Ist v4 deutlich besser als alle anderen?" beantwortet n=3 robust; "ist v6.1 marginal besser als v7.1?" nicht.
- 5. **v6.1-hybrid ist der robusteste Default für Opus 4.7 (no-thinking).** Über RQ-tdd-quality, RQ-tdd-correctness, RQ-context und RQ-stability hinweg landet v6.1 entweder auf Platz 1 oder im engen Spitzenfeld — einziger Workflow ohne Kata-Kollaps und einziger mit 0 % Outlier-Rate auf `cognitive_max`. Der Preis: höchster Token-Verbrauch (33 M auf claim-office, Faktor 17 gegenüber v8a). Für Throwaway-Code bleiben v8a/v8b 5–17× günstiger bei `verification_pct` ≈ 0.97.

6. Limitierungen

- **Anthropic-only:** nur Opus 4.6/4.7, Sonnet 4.6 und Haiku 4.5 — Cross-Provider-Transfer (GPT, Gemini, lokale Modelle) offen.
- **Synthetische TypeScript-Katas** mit ~30–320 LoC Library/CLI-Code; Web-Apps, Datenbank-Code, Async-Systeme, große Codebasen außerhalb des Scopes.
- **Headless ohne HITL** — gemessen wird Vollautonomie. Mensch-im-Loop-Effekte (Resume nach silent-drops, manuelle Code-Reviews zwischen Cycles) sind nicht erfasst.
- **n meist 3–10 pro Zelle;** Tail-Quantile (P95, P99) für v4/v5-Token-Streuung und v4- `mutation_score` -Schwanz nicht belastbar.
- **Code-Qualitäts-Signal nur auf game-of-life, claim-office und claim-office-lite** belastbar — Trivial-Katas (string-calculator, pixel-art-scaler) saturieren auf `smell_total` = 0. mars-rover als zweite Kata bisher nur in prose-Runs erhoben.
- **Coverage-Lücken:** RQ-prompt-correctness bei 22/24 Zellen (zwei Opus-4-7-Zellen ohne Replikate), RQ-stability bei 5/6 Zellen.
- **controls.model:** `{any: [...]}` -**OR-Match** (Portkey + Direct desselben Modells) wird als routing-neutral angenommen; eine systematische Validierung des Routings als unabhängige Achse steht aus — insbesondere für den RQ-model-novel-Befund (Opus 4.6 Portkey vs. Opus 4.7 Direct).
- **Goodhart-Risiko:** Metriken, die in den Refactor-Subagent-Prompts namentlich erwähnt werden (`code_mass` , Funktionsgröße), sind für die strikten Workflows Compliance-Signale. `mutation_score` als prompt-fremder Outcome bleibt der härtere Test (siehe RQ-stability F-stability.7).

7. Reproduzierbarkeit

Alle Daten und Analyse-Skripte liegen im Repo:

- `research/questions/*/README.md` und `research/workflow-dev/*/README.md` — RQ-Definitionen (Frontmatter mit factors/controls/outcomes)
- `research/{questions,workflow-dev}/*/findings.md` — persistente Befund-Listen
- `experiments/runs/*/metrics.json` — Rohdaten pro Run
- `experiments/aggregate-by-query.py` — RQ-spezifische Aggregation
- `experiments/batch-plan-from-rq.py` — Batch-Plan-Generierung aus RQ-Frontmatter
- `experiments/docker/Dockerfile` + `run-batch.sh` + `batch.sh` — Container-Pipeline
- `experiments/analyze-run.sh` + `analyze_transcript.py` — Run-Analyse

Container-Pin: `@anthropic-ai/claude-code@2.1.146` (siehe `experiments/docker/Dockerfile`).

8. Files

Pfad	Inhalt
<code>research/questions/1.1-prompt-style-correctness/findings.md</code>	RQ-prompt-correctness — Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?
<code>research/questions/1.1-prompt-style-correctness/runs.csv</code>	RQ-prompt-correctness aggregierte Run-Metriken
<code>research/questions/1.2-prompt-style-known-kata/findings.md</code>	RQ-prompt-known-kata — Beeinflusst der Prompt-Stil bei einer trainingsbekannten Kata Korrektheit und Code-Qualität?
<code>research/questions/1.2-prompt-style-known-kata/runs.csv</code>	RQ-prompt-known-kata aggregierte Run-Metriken
<code>research/questions/2.1-model-effect-code-quality/findings.md</code>	RQ-model-quality — Wie stark unterscheiden sich die Modelle in der Code-Qualität auf einer trainingsbekannten Kata?
<code>research/questions/2.1-model-effect-code-quality/runs.csv</code>	RQ-model-quality aggregierte Run-Metriken
<code>research/questions/2.2-model-effect-novel-kata/findings.md</code>	RQ-model-novel — Wie unterscheiden sich Opus 4.7 und Opus 4.6 (no-thinking) auf einer novel Kata?
<code>research/questions/2.2-model-effect-novel-kata/runs.csv</code>	RQ-model-novel aggregierte Run-Metriken
<code>research/questions/3.1-workflow-model-interaction/findings.md</code>	RQ-workflow-model — Hängt die Güte eines TDD-Workflows vom Modell ab?
<code>research/questions/3.1-workflow-model-interaction/runs.csv</code>	RQ-workflow-model aggregierte Run-Metriken

Pfad	Inhalt
research/questions/4.1-tdd-effect-code-quality/findings.md	RQ-tdd-quality — Wie wirkt sich die Workflow-Struktur auf die Code-Qualität aus?
research/questions/4.1-tdd-effect-code-quality/runs.csv	RQ-tdd-quality aggregierte Run-Metriken
research/questions/4.2-tdd-effect-correctness/findings.md	RQ-tdd-correctness — Unterscheidet sich die externe Korrektheit zwischen TDD-Workflow-Varianten auf claim-office?
research/questions/4.2-tdd-effect-correctness/runs.csv	RQ-tdd-correctness aggregierte Run-Metriken
research/questions/4.3-tdd-context-engineering/findings.md	RQ-context — Welche Form der Kontext-Strukturierung führt zu besserer Code-Qualität?
research/questions/4.3-tdd-context-engineering/runs.csv	RQ-context aggregierte Run-Metriken
research/questions/5.1-workflow-stability/findings.md	RQ-stability — Wie stabil sind Code-Qualität und TDD-Disziplin pro Workflow?
research/questions/5.1-workflow-stability/runs.csv	RQ-stability aggregierte Run-Metriken
experiments/runs/	Alle Run-Verzeichnisse mit Source, Transcript, Metriken